

Universidad Nacional del Comahue

Facultad de Informática

Conceptos Avanzados de Lenguajes de Programación

2024



Trabajo Práctico Obligatorio Nº2

Manuel Felipe Triñanes

manuel.trinanes@est.fi.uncoma.edu.ar

FAI-2738

Noviembre 2024

Índice

1. Ejercicio 1 **3**
1.1. Excepciones 4
1.2. Flujo de Ejecución 5
1.3. Estados de Pila 6
1.4. Salida 6

2. Ejercicio 2 **7**
2.1. Heap en Rust 7
2.2. Ownership en Rust 8
2.3. Borrowing 9
2.4. Garbage Collector 9

3. Ejercicio 3 **10**
3.1. Tombstones y Locks 10
3.1.1. Tombstone (Scott, 2000, p.392) 10
3.1.2. Locks and Keys (Sebesta, 2012, p.298) 10
3.2. Gestion de memoria Dinamica 11
3.2.1. Algoritmo de conteo de referencias 11
3.2.2. Algoritmo de Mark and Sweep 12
3.2.3. Algoritmo de marcado tricolor 12

4. Ejercicio 4: Scripting **14**

Referencias **14**

1. Ejercicio 1

```
int g[3], m, n, p;

int main () {
    m = 3; n = 1;
    g[0] = 5; g[1] = 4; g[2] = 2;

    try { // externo
        printf("m, n, g[0] = ", m, n, g[0]);
        try { // interno
            printf(m, fun1(m), g[p], g[m]);
        }
        catch (ExcepcionVariableNoInicializada) {
            p = 2;
            printf("p = ", p);
        }
        catch (ExcepcionIndiceFueraDeRango) {
            p = g[0] / g[m];
        }
    }
    catch (ExcepcionVariableNoInicializada) {
        p = 0;
        printf("g[p] = ", g[p]);
    }
    catch (ExcepcionDivisionPorCero) {
        printf(m, n, p);
        n = n - 1;
    }
    catch (ExcepcionIndiceFueraDeRango) {
        m = m - 1;
        printf("g[m] = ", g[m]);
        p = g[0] / g[m];
    }
    finally {
        printf("n, g(m) en finally externo = ", n, g[m]);
    }

    m = n - p;
    printf("m, n, p = ", m, n, p);
}

int fun1(int a) {
    try {
        a = g[a];
        printf("fun1 g(a, n, p) = ", g[a], g[n], g[p]);
    }
    catch (ExcepcionDivisionPorCero) {
        a = a + 1;
        printf(a, n, p);
    }
```

```
50      catch (ExcepcionIndiceFueraDeRango) {
51          a = (-1 * a) + 4;
52          p = 1;
53          printf(a, p);
54      }
55      catch (ExcepcionVariableNoInicializada) {
56          g[a] = 0;
57          printf("a, p = ", a, p);
58      }
59      finally {
60          printf("en fun1: 2a, 2p = ", a + a, p * 2);
61      }
62
63      return (a + 1);
64  }
```

Listing 1: Codigo provisto para el ejercicio 1

1.1. Excepciones

Analisis de las excepciones del programa:

1. **Main:** se tiene un try con 3 catch (**ExcepcionVariableNoInicializada**, **ExcepcionDivisionPorCero**, **ExcepcionIndiceFueraDeRango**) y dentro de dicho try se tiene otro try con 2 catch (**ExcepcionIndiceFueraDeRango**, **ExcepcionVariableNoInicializada**). La linea numero 10 $printf(m, fun1(m), g[p], g[m])$;, definida dentro del try interno, da excepción:

   - **ExcepcionVariableNoInicializada:** Puede dar error ya que la variable p no fue inicializada. Por lo tanto, esta excepción puede ser atrapada por el catch de la linea 12.
   - **ExcepcionIndiceFueraDeRango:** suponiendo que la variable p hubiese sido inicializada, se obtiene otra excepción, ya que el arreglo g (de tres posiciones) tiene como posiciones validas a 0, 1 y 2, pero $m = 3$, luego $g[m] \Rightarrow g[3]$ da excepción por indice fuera de rango. Por lo tanto, esta excepción es atrapada por el catch de la linea 16. La cual vuelve a acceder a $g[m] \Rightarrow g[3]$ generando una excepción la cual va a ser capturada por el catch externo de la linea 28.

2. **Meotodo 'fun1':** En este metodo, llamado por el main en la linea 10 con un $a = 3$, se tiene un try, 3 catch y un finally. Y produce excepciones, las cuales al ser capturadas, no las propagan al metodo llamador, en este caso, el main. Dichas excepciones son:

- **ExcepcionIndiceFueraDeRango**: similar a lo comentado anteriormente. El arreglo g, de tres posiciones, tiene como posiciones validas a 0, 1 y 2. Pero el main llama a fun1 con $a = 3$, produciendo así que la linea 43 al hacer $g[a] \Rightarrow g[3]$ de excepción. Dicha excepción es atrapada por el catch de la la linea 50.
- **ExcepcionVariableNoInicializada**: por mas que la linea 43 no de excepción, la linea 44, al hacer $g[p]$ y p al no estar inicializada, genera una excepción la cual es capturada por la linea 55.

1.2. Flujo de Ejecución

El flujo de ejecución es el siguiente:

1. **Main:**
   - 1.1. **Linea 4**: g = [5, 4, 2], m = 3, n = 1
   - 1.2. **Linea 8**: printf("m, n, g[0] = ", m, n, g[0]);
     Salida: "m, n, g[0] = 3, 1, 5"
   - 1.3. **Linea 10**: printf(m, fun1(m), g[p], g[m]); = printf(3, fun1(3), g[undefined], g[3]);
     Por lo tanto, llama a Fun1 con un $a = 3$
2. **Fun1:**
   - 2.1. **Linea 43:** $a = g[3] \Rightarrow$ ExcepcionIndiceFueraDeRango
   - 2.2. **Linea 50:** captura la excepción
   - 2.3. **Linea 51:** a = (-1 * a) + 4 = (-1 * 3) + 4 = 1
   - 2.4. **Linea 52:** p = 1
   - 2.5. **Linea 53**: printf(a, p)
     Salida: "1, 1"
   - 2.6. **Linea 59**: printf('en fun1: 2a, 2 p = ' , a + a , p * 2); ⇒ printf('en fun1: 2a, 2p = ', 2, 2);
     Salida: 'en fun1: 2a, 2p = 2, 2'
   - 2.7. **Linea 63**: Retorna $a + 1 = 2$
3. **Main:**
   - 3.1. **Linea 10:** Después del llamado a fun1, hace g[p] = g[2] = 3 y hace g[m] con m = 3 ⇒ ExcepcionIndiceFueraDeRango.

3.2. **Lineas 16-18: Catch Interno**
Captura la excepción y hace:
p = g[0] / g[m], otra vez haciendo g[m] con $m = 3 \Rightarrow$ ExcepcionIndiceFueraDeRango.

3.3. **Lineas 28-31: Catch Externo**
Captura la excepción y hace: $m = 3 - 1 = 2$
$p = g[0]/g[m] = 5/g[2] = 5/2 = 2$
Salida: "$g[m] = 2$";

3.4. **Linea 33:** printf("n, g[m] en finally externo =", n, g[m])
Salida: "n, g[m] en finally externo = 1, 2"

3.5. **Linea 38:**
$m = n - p = 1 - 2 = -1$
printf("m, n, p = ", m, n, p)
Salida: "m, n, p = -1, 1, 2"

1.3. Estados de Pila

Fun1	Registros de Activación	
p	Variable Local	1
n	Variable Local	1
m	Variable Local	3
g[2]	Variable Local	2
g[1]	Variable Local	4
g[0]	Variable Local	5

Cuadro 1: Registros de Activación de main.

a	Parametro	3
	Link Dinamico	
	Link Estatico	
	Return Main	

Cuadro 2: Registros de Activación de Fun1.

1.4. Salida

m, n, g[0] = 3, 1, 5 *(Linea 8)*
1, 1 *(Linea 53)*
en fun1: 2a, 2p = 2, 2 *(Linea 60)*

p = 2 *(Linea 14)*
g[m] = 2 *(Linea 30)*
n, g[m] en finally externo = 1, 2 *(Linea 34)*
n, m, p = -1, 1, 2 *(Linea 38)*

2. Ejercicio 2

Enunciado

Investigar y Describir cómo se administra la memoria Heap en el lenguaje RUST. Denir y explicar el concepto de Propiedad (Ownership) y de que manera soluciona las referencias sueltas y garbage. Ejemplique.

El lenguaje de programación Rust ha ganado una considerable popularidad en los últimos años debido a su enfoque en la seguridad, el rendimiento y la concurrencia sin sacrificar la eficiencia. Rust ofrece características que lo hacen especialmente adecuado para aplicaciones de bajo nivel, sistemas embebidos, y el desarrollo de software de alto rendimiento, como videojuegos o sistemas operativos. Una de las características que lo distingue de otros lenguajes de sistemas es su modelo de gestión de memoria basado en los conceptos de ownership (propiedad) y borrowing (préstamo), que permiten la manipulación segura de la memoria sin necesidad de un recolector de basura.

En esta sección ampliaremos sobre dichos conceptos junto con ejemplificaciones de los mismos.

2.1. Heap en Rust

En Rust, los datos asignados a memoria heap son gestionados a través de un puntero inteligente llamado Box. Un Box es una forma de almacenar valores en el heap. Cuando un valor es almacenado dentro de un Box, Rust garantiza que el valor será limpiado (liberado) cuando el Box salga de su ámbito. Esta es una forma en que Rust evita los problemas de memoria típicos de otros lenguajes, como fugas de memoria o referencias colgantes.

Se puede realizar la:

- **Asignación**: al usar Box::new para asignar memoria en el heap.
- **Liberación**: mediante un sistema de gestión automática de memoria que no requiere un *garbage collector*. Cuando el Box deja de estar en uso, Rust automáticamente libera la memoria asociada con él.

```rust
fn main() {
    let b = Box::new(5);
    println!("{}", b);
}
```

Listing 2: Ejemplo de asignacion de Heap

En este ejemplo, se asigna el valor 5 en heap, el cual luego es impreso. Una vez que la variable 'b' sale del alcance, se libera la heap automaticamente.

2.2. Ownership en Rust

El concepto de Propiedad (Ownership) es una de las características más distintivas y poderosas de Rust. Se refiere a cómo se gestionan los recursos (como la memoria) en el lenguaje. El sistema de propiedad asegura que no haya errores como referencias colgantes o fugas de memoria, eliminando la necesidad de un *garbage collector*.

Reglas de Propiedad:

- Cada valor tiene un único propietario: Solo una variable puede ser dueña de un valor en un momento dado. Esto garantiza que la memoria se libere de manera segura cuando el valor deje de existir.
- Cuando el propietario se mueve, el valor se transfiere: Cuando un valor es asignado a otra variable o pasado a una función, el valor se mueve. Esto significa que la variable original deja de tener acceso al valor.
- Cuando un valor sale del alcance, se destruye: Cuando el propietario de un valor sale del alcance, el valor se libera automáticamente, eliminando el riesgo de fuga de memoria.

```rust
fn main() {
    let s1 = String::from("Hola");
    let s2 = s1;

    println!("{}", s2);      // Imprime "Hola"
    println!("{}", s1);  // Error: 's1' ya no es valido
}
```

Listing 3: Ejemplo de ownership de Rust

En este ejemplo, cuando s1 se mueve a s2, ya no se puede acceder a s1, ya que el valor ha sido 'traspasado' a s2. Esta regla asegura que la memoria no sea liberada accidentalmente mientras aún haya referencias al valor.

2.3. Borrowing

En Rust, se pueden tomar referencias a valores en lugar de mover la propiedad. Existen dos tipos de referencias:

- **Referencias inmutables (&T):** Permiten leer un valor sin modificarlo.
- **Referencias mutables (&mut T):** Permiten modificar el valor, pero solo puede existir una referencia mutable a la vez.

Las referencias resuelven el problema de las 'referencias colgantes' (*dangling references*) que se producen cuando una variable se refiere a una dirección de memoria que ya ha sido liberada. En Rust, no es posible tener una referencia a un valor que haya sido movido o liberado, evitando así referencias colgantes.

```
fn main() {
    let s = String::from("Hola");

    let r1 = &s;
    let r2 = &s;
    println!("{}", r1);
    println!("{}", r2);

    let r3 = &mut s;
}
```

Listing 4: Ejemplo de borrowing en Rust

En este caso, r1 y r2 son referencias inmutables a s. Como son inmutables, se pueden tener varias al mismo tiempo. Si intentamos crear una referencia mutable (r3), el compilador generará un error porque no se puede tener una referencia mutable y una inmutable al mismo tiempo.

2.4. Garbage Collector

Rust no utiliza un *garbage collector* tradicional. En lugar de eso, su sistema de propiedad asegura que los recursos sean liberados de manera automática y determinista cuando ya no son necesarios. Las referencias sueltas (*dangling pointers*) son imposibles en Rust debido a su sistema de propiedad. Cuando una variable sale de su ámbito, el valor asociado se elimina automáticamente, y no pueden quedar referencias huérfanas".

Algunas de las ventajas de este enfoque son:

- **Sin coste en tiempo de ejecución**: Al eliminar la necesidad de un recolector de basura, Rust elimina el overhead que otros lenguajes tienen.
- **Seguridad en memoria**: Los errores comunes como referencias colgantes y fuga de memoria son evitados gracias al sistema de propiedad y préstamos.

3. Ejercicio 3

Enunciado

Dar la denición de los siguientes algoritmos y ejemplicar con este esquema de memoria.

1. *tombstones* y *locks ands keys*.
2. Gestión de memoria dinámica y garbage collector:
   a) Algortimo de conteo de referencias
   b) Algoritmo de Mark and Sweep
   c) Algoritmo de marcado tricolor.

3.1. Tombstones y Locks

3.1.1. Tombstone (Scott, 2000, p.392)

Un tombstone, lapida en español, es una estructura que actúa como intermediaria entre el puntero y el objetivo. El puntero solo apunta a las lapidas y nunca al objetivo actual. Cuando el dato es desalojado, la lapida se setea como null, o, generalmente a un valor que no es valido para un puntero, indicando que la variable ya no existe.

Este mecanismo previene el uso de punteros inválidos, los cuales de otro modo accederían al área de memoria que antes pertenecía a la variable ahora desalojada, aunque esa área ya pueda contener otros datos, lo que a su vez conduciría a la corrupción de los datos en memoria.

Ejemplo: Si se libera el bloque 8, el cual es referenciado a través de un tombstone por r2, el tombstone correspondiente indicaria que el bloque 0 ya no existe, evitando asi un error de acceso por parte de r2.

3.1.2. Locks and Keys (Sebesta, 2012, p.298)

La tecnica de locks-and-keys representa un puntero como un par odenado [llave, direccion] donde la llave es un valor entero. Las variables se representan como el almacenamiento para la variable más una celda para un valor entero de la cerradura. Cuando una variable es alocada, el valor de la cerradura es creado y colocado tanto en la celda de la variable como en el campo de *llave* del puntero. Cada acceso al puntero compara dichos valores (llave y cerradura) y el acceso es concedido solo si ambas son iguales.

Cuando se desaloca una variable, la llave de su puntero es modificada para contener un valor diferente al de la celda de la variable. A partir de ese

momento, cualquier intento de desreferenciar el puntero puede ser marcado como un error. Dado que copiar un puntero también copia su valor de celda, cambiar la llave del par ordenado desactiva de manera segura todas las copias del puntero.

Ejemplo: Si se libera el bloque 8, el cual es referenciado por r2 mediante un lock-and-key, la llave correspondiente ya no coincidiría con la cerradura del recurso 'reteniendo' el acceso de r2 al recurso liberado.

3.2. Gestion de memoria Dinamica

3.2.1. Algoritmo de conteo de referencias

Este es un enfoque sencillo para la gestión de memoria en el que cada objeto tiene un contador que mantiene un registro del número de referencias (o punteros) que apuntan a ese objeto.

Cada vez que se crea una nueva referencia a un objeto, el contador se incrementa. Cada vez que se elimina una referencia, el contador se decrementa. Si el contador de un objeto llega a cero, significa que ya no hay referencias que apunten a él, por lo que el recolector de basura puede liberar esa memoria.

Este algoritmo, tiene la ventaja de ser sencillo y cumplir con su cometido, pero tiene como desvetaja que no es capaz de detectar referencias cíclicas. Es decir, si dos o más objetos se refieren mutuamente pero no son accesibles desde fuera de este ciclo, el contador nunca llegará a cero y la memoria no se liberará.

Siguiendo el esquema de heap dado:

N de Bloque	Cant. Referencias
0	1
1	0
2	0
3	0
4	1
5	2
6	2
7	3
8	1
9	1
10	2
11	1

Esta implementación haría que el garbage collector se encargue de los bloques 1, 2 y 3. Sin embargo, en el caso de los bloques 5 y 11, que se referencian mutuamente, el bloque 5 seguiría teniendo una referencia por parte del bloque 11, y el bloque 11 seguiría siendo referenciado por el bloque 5. Por lo tanto,

aunque se elimine el bloque 1 (que referencia a 5), el garbage collector no se ocuparía de los bloques 5 y 11, ya que ambos seguirían siendo accesibles a través de la referencia mutua entre ellos.

3.2.2. Algoritmo de Mark and Sweep

El algoritmo de Mark and Sweep (marcar y barrer) mejora el algoritmo de conteo de referencias, especialmente al tratar con referencias cíclicas. Se compone de dos fases: la **fase de marcado** y la **fase de barrido**.

En la **fase de marcado**, el recolector de basura recorre todos los objetos accesibles desde las raíces del programa, como las variables globales o los elementos de la pila, y marca esos objetos como alcanzables. Luego, en la **fase de barrido**, el recolector recorre toda la memoria y libera aquellos objetos que no han sido marcados, es decir, que no son accesibles desde las raíces del programa.

El algoritmo de Mark and Sweep resuelve el problema de las referencias cíclicas, ya que puede liberar objetos que están involucrados en ciclos de referencias, pero que no son accesibles desde las raíces. Sin embargo, este algoritmo puede ser costoso en términos de tiempo y recursos, ya que requiere dos pasadas sobre la memoria: una para marcar y otra para barrer.

Siguiendo al esquema dado:

1. Realiza un recorrido comenzando de punteros de afuera del heap marcando a los bloques como en uso:
   a) Partiendo de $r1 \to 7 \to 9 \to 4$, se marcan como en uso a los bloques 4, 7 y 9
   b) Partiendo de $r1 \to 8 \to 10 \to 6$, se marcan como en uso a los bloques 6, 8 y 10
2. Se hace un barrido ("*sweep*") de los bloques no marcados como en uso, en otras palabras, los no recorridos en 1a y 1b. Por lo tanto se barren los bloques 0, 1, 2, 3, 5 y 11.

De esta forma, esta implementación solucionó la referencia ciclica entre 5 y 11 en la cual falla el algoritmo de conteo de referencias.

3.2.3. Algoritmo de marcado tricolor

El algoritmo de marcado tricolor es una mejora sobre el algoritmo de Mark and Sweep, diseñado para optimizar el proceso de recolección de basura en sistemas con referencias complejas. La idea principal de este algoritmo es clasificar los objetos en tres categorías, utilizando tres colores: blanco, gris y negro.

En este enfoque, los objetos inicialmente se marcan como blancos, lo que significa que aún no han sido visitados. Los objetos accesibles desde las raíces se marcan como grises. Luego, los objetos grises se examinan, y sus referencias se marcan como negras, lo que indica que han sido completamente visitados. Los objetos que permanecen blancos después de este proceso son aquellos que no han sido alcanzados desde las raíces y, por lo tanto, pueden ser considerados basura y ser liberados.

Este algoritmo es más eficiente que el tradicional Mark and Sweep, ya que evita la necesidad de recorrer toda la memoria en la fase de barrido. Además, permite detectar ciclos y objetos inaccesibles de manera más rápida. No obstante, la complejidad del algoritmo puede aumentar dependiendo del comportamiento del sistema y el número de referencias en los objetos.

Siguiendo el ejemplo dado:

1. Coloca todos los bloques en el conjunto blanco:
   - Blanco: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
   - Gris: ...
   - Negro: ...

2. Cambia los bloques referenciados en blanco a gris:
   - Blanco: 0, 1, 2, 3, 4, 5, 6, 9, 10, 11
   - Gris: 7, 8
   - Negro: ...

3. Cambia los bloques de gris a negro (7 y 8) y de blanco a gris los que son referenciados por los bloques anteriormente en gris (9 y 10):
   - Blanco: 0, 1, 2, 3, 4, 5, 6, 11
   - Gris: 9, 10
   - Negro: 7, 8

4. Cambia los bloques de gris a negro (9 y 10) y de blanco a gris los que son referenciados por los bloques anteriormente en gris (4 y 6):
   - Blanco: 0, 1, 2, 3, 5, 11
   - Gris: 4, 6
   - Negro: 7, 8, 9, 10

5. Cambia los bloques de gris a negro y de blanco a gris los que son referenciados por los bloques anteriormente en gris:
   - Blanco: 0, 1, 2, 3, 5, 11
   - Gris: ...
   - Negro: 4, 6, 7, 8, 9, 10

6. Luego, como no hay bloques en gris, se barren los bloques en blanco y detiene el algoritmo.

4. Ejercicio 4: Scripting

En mi caso particular, me siento más cómodo utilizando Python. Esto se debe a que, ademas de su simplicidad para lograr algoritmos como este, es el lenguaje que más utilizo en el trabajo que tengo actualmente.

```python
import os
from PIL import Image

INPUT_DIR = '.'                 # Obtenemos las imagenes del directorio actual, el escritorio
OUTPUT_DIR = './thumbnails'     # Guardamos los thumbnails en el subdirectorio thumbnails

def crear_thumbnails(input_dir, output_dir,tamanio=(128, 128)):
    for archivo in os.listdir(input_dir):
        if archivo.lower().endswith('.jpg'):
            ruta_archivo = os.path.join(input_dir, archivo)
            with Image.open(ruta_archivo) as img:
                img.thumbnail(tamanio)
                img.save(os.path.join(output_dir, f"thumb_{archivo}"))

crear_thumbnails(INPUT_DIR, OUTPUT_DIR)
```

Listing 5: Script en Python para generar Thumbnails

Referencias

Scott, M. L. (2000). *Programming language pragmatics*. San Francisco: Morgan Kaufmann.

Sebesta, R. (2012). *Concepts of programming languages* (10th ed.). Boston: Pearson.